

MERN Full Stack — Complete Roadmap 2026

Sheriyans Coding School Playlist + What To Do After It | Industry-Ready Goal

For: Knows React · Now learning backend · New to Git · Goal: Job-ready MERN developer

How this roadmap works: Part 1 covers what to do BEFORE and DURING the Sheriyans playlist (Phases 0–1). Part 2 is the Sheriyans playlist itself with what to build at each stage (Phases 2–6). Part 3 is everything AFTER the Sheriyans playlist — the 5 missing topics that make you industry-ready (Phases 7–11). Part 4 is the final portfolio project. Total time: **14–16 weeks at 90 min/day**.

ROADMAP AT A GLANCE

Part	Phases	What You Do	Weeks
Part 1 — Before Sheriyans	0–1	Git/GitHub + JS async refresh	1
Part 2 — Sheriyans Playlist	2–6	Node, Express, MongoDB, Auth, Uploads, Projects	7
Part 3 — After Sheriyans	7–11	React↔Backend, Deployment, Security, Testing	5
Part 4 — Portfolio Project	12	Full MERN app: live, tested, documented	2–3

SHERIYANS COURSE — WHAT IT COVERS (You don't need extra resources for these)

- ✓ **JavaScript async, promises, array methods** — solid foundation
- ✓ **How the internet works** — TCP/IP, DNS, HTTP, VPN, proxy, reverse proxy
- ✓ **Node.js core** — event loop, non-blocking I/O, NPM, file system, HTTP module
- ✓ **Express.js** — routing, middleware, CORS, cookies, sessions, Morgan, static files
- ✓ **EJS + SSR + form handling** — understand SSR concept, but React is your real frontend
- ✓ **MongoDB + Mongoose** — CRUD, schemas, all query operators, aggregation, relationships
- ✓ **JOI validation** — professional input validation on every route
- ✓ **JWT authentication** — register, login, protected routes, role-based access
- ✓ **Multer file uploads** — single/multiple files, type validation, disk storage
- ✓ **Redis caching** — BONUS, most free courses don't cover this
- ✓ **WebSockets + Socket.IO** — BONUS, real-time chat example
- ✓ **Khatabook project** — real file-based project for practice

WHAT THE COURSE DOES NOT COVER — Fill these after finishing the playlist

- ✗ **Git & GitHub** — Learn this BEFORE starting the playlist. Zero exceptions.
- ✗ **React ↔ Backend integration** — CORS in React, Axios, JWT in frontend, protected routes
- ✗ **Deployment** — Railway/Render + Vercel + MongoDB Atlas — make your projects live
- ✗ **API Security** — rate limiting, Helmet.js, secure cookies (CORS is there but that's all)
- ✗ **Testing** — Jest + Supertest API tests — expected in almost every backend job

PART 1 — DO THIS BEFORE STARTING THE SHERIYANS PLAYLIST

Phase 0 Git & GitHub

Week 1 · Days 1–3

- Install Git. Learn: init, clone, add, commit, status, log, diff.
- Branching: branch, checkout, merge. Understand why branches exist.
- GitHub: create account, push your first repo, write a proper README.
- .gitignore — add node_modules and .env before your first commit. Always.
- Pull requests: how they work, why teams use them.

■ **Video:** Git & GitHub Crash Course 2025 — freeCodeCamp

youtube.com/watch?v=vA5TTz6BXhY

■ *Git is tested in every developer interview. Employers look at your GitHub commit history. Start now.*

■ **CHECKPOINT:** Push a small folder to GitHub with a README and .gitignore. That is your proof of concept.

Phase 1 JavaScript Async — Quick Refresh

Week 1 · Days 4–5

- Promises: .then(), .catch(), chaining, Promise.all().
- async/await: how it works, try/catch with async functions.
- CommonJS modules: require() vs ES modules import/export — backend uses require() by default.
- Destructuring, spread operator, optional chaining — you use these in React already.

■ *Since you know React, most of this is familiar. The only real difference on the backend is CommonJS modules (require vs import).*

■ **CHECKPOINT:** Write 5 async functions: read a fake API, handle errors, run two requests in parallel with Promise.all.

PART 2 — THE SHERIYANS PLAYLIST (Videos 1–29 in order)

- Node.js: event loop, non-blocking I/O, how it handles requests without threads.
- NPM: install packages, package.json scripts, devDependencies vs dependencies.
- File system module: reading/writing files async — fs.readFile, fs.writeFile.
- Express: create server, routing, route params, query params, status codes.
- Middleware: what it is, writing your own, chaining multiple middleware.
- Sessions, cookies, CORS, cookie-parser, Morgan — all covered in the playlist.
- EJS and server-side rendering — learn the concept but remember: React is your frontend.
- dotenv: store all secrets in .env — never hardcode database URLs or passwords.

■ *EJS is for SSR projects. Since you're building MERN, you'll skip EJS in your actual projects — but understanding how SSR works is useful knowledge.*

■ **CHECKPOINT: Build a Notes API with in-memory data: GET all, GET one, POST, PUT, DELETE. Test every route in Postman. Push to GitHub.**

- MongoDB: documents, collections, BSON. Why NoSQL over SQL for this stack.
- Mongoose: connect to DB, define schemas, create models.
- CRUD: create, find, findById, findByIdAndUpdate, findByIdAndDelete.
- Schema validation: required, unique, min, max, enum, custom validators.
- Advanced queries: \$eq, \$gt, \$lt, \$in, \$nin, \$and, \$or, \$regex, \$exists.
- Relationships: embedding vs referencing (ObjectId + populate).
- Aggregation: \$match, \$group, \$sort, \$project, \$lookup — powerful data queries.
- Joi validation: validate every incoming request body before it touches the database.

■ *Separation matters: keep DB logic in separate files from route files. Routes call service functions, service functions call models.*

■ **CHECKPOINT: Blog API: users, posts, comments. Each post has an author (ObjectId reference). GET /posts has pagination and title search. Push to GitHub.**

- bcrypt: hash passwords on register, compare on login. Never store plain text.
- JWT: sign token on login, verify token in middleware, protect routes.
- Auth flow: register → hash → save. Login → compare → send JWT.
- Role-based access: admin routes vs user routes, middleware checks role.
- Refresh token concept: why they exist, how to implement a basic version.
- Multer: upload single file, multiple files, validate file type and size.
- Khatabook project + mini project: follow along and build every line yourself.
- Production setup: folder structure (routes, controllers, models, middlewares, utils).

■ *By the end of Phase 4 you have a complete Node+Express+MongoDB backend. The next phases make it actually usable as a MERN developer.*

■ **CHECKPOINT: E-commerce backend: products with image uploads, categories, cart, orders, JWT auth, admin-only add/delete. Push to GitHub.**

PART 3 — AFTER THE SHERIYANS PLAYLIST (The 5 gaps that make you industry-ready)

You finished the Sheriyans playlist. You have a working backend. But right now your React and your backend are two separate things that have never talked to each other. Part 3 fixes that — and adds deployment, security, and testing. These 5 phases take 4–5 weeks.

Phase 5 React ↔ Backend Integration

Week 9 | Most Important Phase

- CORS: configure Express to allow requests from your React app's domain.
- Axios in React: set baseURL, make GET/POST/PUT/DELETE calls to your Express API.
- Axios interceptors: automatically attach JWT token to every request header.
- Handle 401 (unauthorized): redirect to login page when token expires.
- Store JWT: understand the tradeoff — localStorage (easy, XSS risk) vs httpOnly cookies (safer).
- Protected routes in React: check if token exists before rendering a page.
- Environment variables in React: use VITE_API_URL to point to your backend URL.
- Proxy setup: add proxy in Vite config so React calls go to Express in development.
- Test the full flow: register in React → data saved in MongoDB → login → see protected page.

■ **Video:** How to Create React Frontend + Express Backend —
Jan 2025

youtube.com/watch?v=yc_D8u7ETuw

■ *This is the phase most MERN tutorials skip. React calling your API sounds simple — but CORS errors, token handling and proxy setup will stop you cold if you don't learn this properly.*

■ **CHECKPOINT:** Take your auth backend from Phase 4. Build a React frontend: register page, login page, protected dashboard. Full flow working end to end.

- Proper REST resource naming: /api/v1/users not /getUsers or /user_list.
- HTTP methods used correctly: GET fetches, POST creates, PUT/PATCH updates, DELETE removes.
- Status codes you must know: 200, 201, 400, 401, 403, 404, 409, 422, 500.
- API versioning: prefix all routes with /api/v1/ — makes future changes safe.
- Filtering, sorting, pagination via query params: ?page=1&limit=10&sort=createdAt.
- Swagger/OpenAPI: add swagger-ui-express to one project. Document every route with request body and response schema.
- Clean README: what the project does, how to run it, all endpoints listed, example requests.

■ **Video:** Node.js and Express.js Full Course — freeCodeCamp

youtube.com/watch?v=Oe421EPjeBE

■ *Swagger docs in your GitHub repo signals professionalism. Most junior devs skip this. It takes 2 hours and separates your portfolio from the crowd.*

■ **CHECKPOINT: Add Swagger docs to your e-commerce backend. Every route has a documented request body, response, and error example. Update your README.**

- express-rate-limit: block brute-force login attempts — 5 failed attempts = 15 min cooldown.
- Helmet.js: one line that sets 11 secure HTTP headers automatically — always include it.
- Input sanitization: express-validator or Joi to block injection attacks and garbage data.
- Secure cookie settings: httpOnly (no JS access), secure (HTTPS only), sameSite flags.
- Safe error messages: never send stack traces or MongoDB errors to the client in production.
- Secrets in .env only: DB connection string, JWT secret, API keys — never in source code.
- Dependency scanning: run npm audit regularly, fix critical vulnerabilities.

■ **Video:** API Security Fundamentals — freeCodeCamp

youtube.com/watch?v=R-4_DbV1Su4

■ *Security is not advanced — it is basic expectation. These are 3 npm packages (rate-limit, helmet, validator) added in one afternoon.*

■ **CHECKPOINT: Go back to your best project. Add rate limiting, Helmet, input validation, safe error responses. Write a security checklist in the README.**

- MongoDB Atlas: create free cluster, get connection string, whitelist all IPs (0.0.0.0/0).
- Render (backend): connect GitHub repo, set environment variables in dashboard, deploy.
- Vercel (frontend): import React repo from GitHub, set VITE_API_URL to your Render URL.
- Update CORS in Express to allow your Vercel domain in production.
- Test the full live app: register, login, see data — everything working on real URLs.
- Every portfolio project must have a live link — add it to your GitHub README.

■ **Video:** Deploy MERN Stack — Render + Vercel + MongoDB Atlas — March 2026

youtube.com/watch?v=uGH7M2NEu3Q

■ *A deployed project is worth 10x a local one when job hunting. Employers who click your GitHub link and see a live URL take you seriously.*

■ **CHECKPOINT: Your complete MERN app is live: React on Vercel, Express on Render, MongoDB on Atlas. Open the URL on your phone. Share the link in your README.**

- Why testing: catch bugs before users do — required in most backend jobs.
- Jest: write unit tests for utility functions and individual service logic.
- Supertest: send real HTTP requests to your Express routes in tests — no server needed.
- Test your auth routes: register success, login success, wrong password, missing fields.
- Test CRUD: create success, read list, read one, update, delete, item not found (404).
- Test authorization: admin route rejects regular user (403), no token = 401.
- MongoMemoryServer: run tests against a fake in-memory MongoDB — never touch real data.
- Aim for 15+ tests. Coverage % doesn't matter at this stage — variety of test types does.

■ **Video:** Supertest Express Tutorial — API Testing for Node.js

youtube.com/watch?v=T2sYitv2OAY

■ *You don't need 100% coverage. Having any tests at all puts you ahead of 80% of junior applicants. Show employers you understand the concept.*

■ **CHECKPOINT: 15+ tests covering: auth (4 tests), CRUD (6 tests), validation failures (3 tests), authorization (2 tests). All passing.**

PART 4 — FINAL PORTFOLIO PROJECT (Weeks 14–16)

Build ONE complete MERN project that uses everything you learned. This is what you will talk about in every interview. Choose something you are actually interested in — you will be explaining it for months.

Project Idea	Why It's Good for Interviews
Job Board API	Companies post jobs, users apply, admin approves — real-world auth + roles
Learning Management	Courses, lessons, enrollment, progress — complex data relationships
Expense Tracker	Users track expenses, categories, monthly reports — aggregation practice
Appointment Booking	Users book time slots, admin manages availability — scheduling logic
Real-time Chat	Socket.IO already covered in Sheriyans — use that WebSocket knowledge

YOUR FINAL PROJECT MUST INCLUDE ALL OF THESE

- **Full JWT auth** — register, login, logout, role-based access (admin + user)
- **MongoDB with proper schema design** — references between collections, not everything embedded
- **File upload** — profile picture or file attachment using Multer
- **Input validation** — Joi or express-validator on every POST/PUT route
- **Rate limiting + Helmet** — added to the Express app
- **React frontend connected** — Axios with interceptors, protected routes, auth flow
- **15+ API tests** — auth, CRUD, validation, authorization
- **Swagger documentation** — every route documented with request/response schemas
- **Deployed and live** — React on Vercel, Express on Render, MongoDB on Atlas
- **README with: setup steps, architecture diagram, API docs link, live URL, test command**

WEEKLY ROUTINE

Day	What You Do	Time
Mon – Thu	Watch tutorial, code along, build the feature yourself	90–120 min
Friday	Rebuild that week's concept from scratch without watching	60–90 min
Saturday	Add tests, documentation, clean code, push to GitHub	60 min
Sunday	Review what you built, write notes, plan next week	30 min

INTERVIEW PREPARATION — What You Must Be Able To Explain

- **Event loop** — how Node.js handles many requests on one thread
- **Middleware** — write one from scratch on a whiteboard
- **JWT vs sessions** — tradeoffs, when to use each
- **MongoDB vs SQL** — when would you choose MongoDB and why
- **CORS** — what it is, why it blocks requests, how to fix it

- **bcrypt** — why you hash passwords, what the salt rounds do
- **Protected routes** — explain the auth middleware flow end to end
- **Your project** — explain every decision: why this schema, why this route structure
- **Git** — branch, merge, pull request — can you do this live

Bottom Line: Sheriyans is Phase 2–4 of this roadmap — not the whole journey. Before it: learn Git. Inside it: build every project, push everything to GitHub. After it: connect React to your backend (Phase 5), deploy one project live (Phase 8), add security and tests. The integration and deployment phases are what most people skip — and exactly what separates you from the crowd.